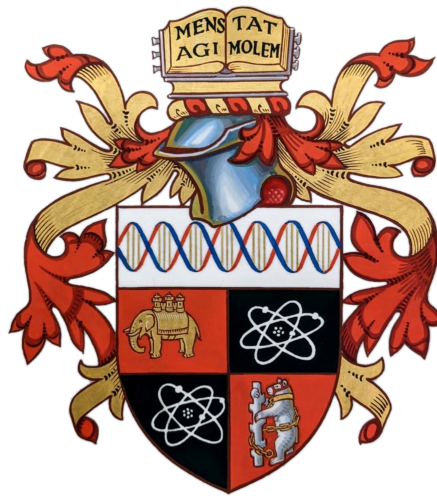




NON-PHOTOREALISTIC RENDERING AND PROCEDURAL CONTENT GENERATION FOR GAMES: AN INDUSTRY-ORIENTED REVIEW

by

SHUNYU YANG

A thesis submitted in partial fulfilment
of the requirements for the degree of

MASTER OF SCIENCE

UNIVERSITY OF WARWICK

WARWICK MANUFACTURING GROUP

12th, April, 2026

CONTENTS

Contents	ii
Declaration	iii
Abstract	iv
List of abbreviations	v
1 Introduction	1
1.1 NPR and PCG Overview	1
1.2 Background	1
1.3 Thesis	2
2 Comparison	4
2.1 NPR	4
2.1.1 Silhouette Extraction	4
2.1.2 Shading	5
2.2 PCG	7
2.2.1 Parametric Methods	7
2.2.2 Rule based	7
3 Discussion	9
3.1 Gap and limitation	9
3.2 Potential directions	9
4 Conclusion	12
Bibliography	13

DECLARATION

This thesis is submitted to the University of Warwick in support of my application for the degree of Master of Science. It has been composed by myself and has not been submitted in any previous application for any degree. The work presented (including data generated and data analysis) was carried out by the author.

ABSTRACT

Non-Photorealistic Rendering (NPR) and Procedural Content Generation (PCG) represent two distinct domains within Computer Graphics (CG) and Games Engineering (GE). NPR focuses on stylized visual representation and is widely adopted in animation style and indie games, whereas PCG aims at the automated generation of diverse game content and plays a significant role in modern game development. Although these domains operate along different technical paths, both can be interpreted as processes of selecting and refining candidates from high-dimensional information, differing primarily in whether the space encodes visual features or game content.

Over the past decades, both fields have developed rapidly, forming a positive feedback loop in which theoretical advances support game creation, while practical applications inform and refine research directions. In this context, this paper aims to examine existing academic research from a game development perspective, and to identify practical and effective technical pathways for applying NPR and PCG in the game. First, this paper outlines the detailed classifications of NPR and PCG. Second, author provides a researches review within two key classifications of both domains in game practice view. Finally, author comprehensively discussed the gap between theory and practice from multiple perspectives, the potential limitations, summarized the achievement of the review objectives, and potential directions for future research and practice.

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
CG	Computer Graphics
CV	Computer Vision
GE	Games Engineering
HCI	Human Computer Interaction
LUT	Look Up Table
NPR	Non-Photorealistic Rendering
PBR	Physically Based Rendering
PCG	Procedural Content Generation
QI	Quantitative Invisibility

1 INTRODUCTION

1.1 NPR AND PCG OVERVIEW

In the field of CG, NPR stands in opposition to Physically Based Rendering (PBR) (realistic rendering). It aims to produce images that are clearly understandable to humans without relying entirely on real physics. While NPR is not the most mainstream rendering method compared to PBR, it still holds a dominant position in indie and small scale games, and is even showing a strong trend of encroaching on mainstream AAA games recently. It emphasizes conveying stylized and abstract image information, enhancing the efficiency of visual information delivery, and reducing the cognitive load on users based on human perception [1]. Besides, it still overlaps with Computer Vision (CV), Human Computer Interaction (HCI), and Artificial Intelligence (AI) to some extent. Beyond its precise academic definition, its practical effects are often considered, extending into the realm of art, making it a more subjective field compared to other areas.

Similarly, PCG also has a high degree of subjectivity and spans multiple areas, however, its core is more located in GE, generating a variety of content that simulates real-world patterns. PCG's main purpose is to use specific computer algorithms to generate constrained content [2], with its most famous application in the game industry. PCG has made significant contributions to game content generation, providing players with content experiences far exceeding the limitations of the number of developers. Compared to NPR, more considerable amount of fields is related to PCG. Regardless of the method used, any pattern that generates a simulated reality will fall under PCG. The content generated by PCG is also highly subjective, making it a discipline that has attracted the attention of many game researchers and designers.

1.2 BACKGROUND

The origins of NPR can be traced back to the work of Appel [3], who proposed one of the earliest contour detection algorithms in 1967. This method could more clearly represent three-dimensional geometry in two-dimensional images and is often considered the early foundation of NPR in CG. In 1986, Strassmann introduced brush-based texture models, opening up the direction of painterly and brush-like rendering [4]. In the early 1990s, with the development of Mista's work [5], stylized image compositing techniques were further improved. Since then, research on NPR has been divided into three main categories as

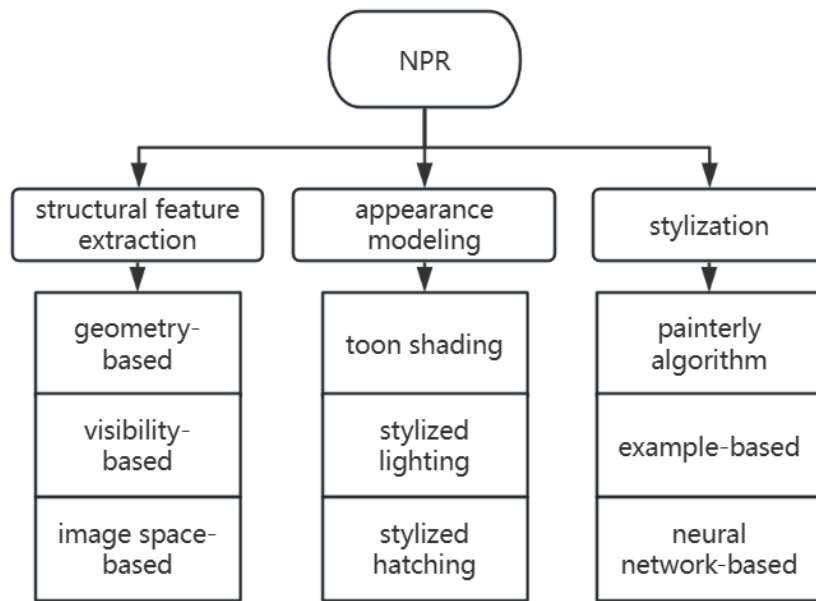


FIGURE 1.1: NPR classification in academic view

figure 1.1.

The situation with PCG is quite unique compared to NPR. Early NPR methods were unable to be integrated into real-time applications due to limitations in algorithm performance, complexity, and hardware. While the earliest PCG methods were proposed by industry. A game called *Rogue* generate its dungeon levels in 1980, igniting academic interest, currently is classified to rule-based generation method. The first PCG method in academia can be traced back to Ken Perlin's noise algorithm in 1985 [6], initially intended for procedural texture research in computer graphics. It wasn't until 2010 that Julian Togelius explicitly proposed the concept of PCG and introduced a subfield: search-based methods [7]. Same year, Doran used a agent-based approach to generate terrain [8]. Then came the publication of *Procedural Content Generation in Games* [2]. This book summarized the relevant techniques in these areas and provided practical examples, summarising and categorizing some past research as grammar-based methods, and also mentioning constraint-based and pattern-based generation methods. Until Summervill demonstrated the feasibility of machine learning in PCG [9], from a technical perspective, PCG can be divided into eight directions as figure 3.3 shown.

1.3 THESIS

This paper will compare and analyze the different technical approaches of NPR and PCG, the two directions with the greatest potential for application in games. It aims to provide reference for game researchers and designers to get started, understand the most popular related technologies in the field, select technologies used in relevant game implementations, understand the theoretical and practical gaps in the field, trace the latest developments in

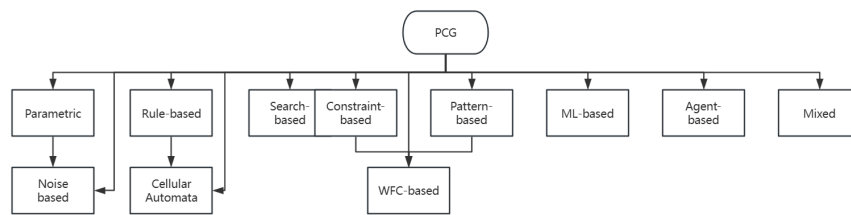


FIGURE 1.2: Classification of PCG, there are some controversial areas and overlapping areas

the field, or explore potential future directions.

2 COMPARISON

2.1 NPR

2.1.1 SILHOUETTE EXTRACTION

The case of classification view of industry differ to aforementioned academic view, where concern lies as figure 2.1. Contour research began with the work of Appel [3], who introduced the concept of Quantitative Invisibility (QI). This concept defines the number of polygons a ray of light passes through from the camera to a point on an object. A change in the QI of the ray is considered a detected contour. Then, McCreynolds used early GPUs for computation, utilizing image spatial information at the same time [10]. It used three rendering passes, employing stencils to record depth data and render the geometry's faces away from the camera. In the same year, Zhang proposed a method for detecting contours using normals [11], as shown in the formula section 2.1.1. Until Decarlo [12] used filter to analyze contours in images, forming basically the main sources of ideas for all modern contour detection, which can be classified into three main methods: geometry-based (Appel's), screen space-based (Zhang and McCreynolds'), and image analysis-based (Decarlo's).

$$(\vec{n}_1 \cdot (\vec{v} - \vec{e})) (\vec{n}_2 \cdot (\vec{v} - \vec{e})) \leq 0$$

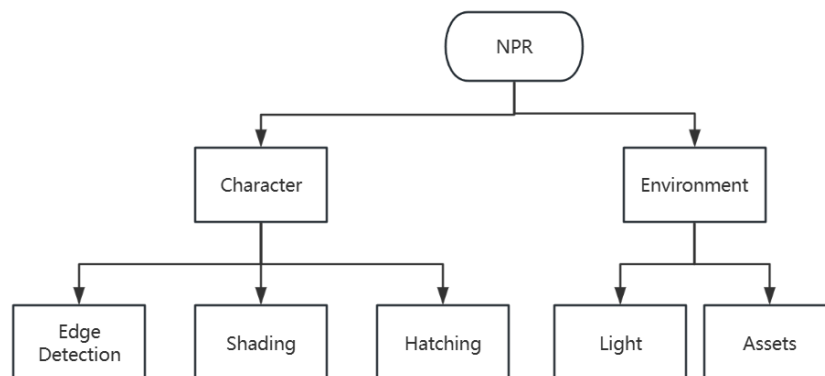


FIGURE 2.1: classification in industrial view

It can be observed that the QI concept's ray tracing approach, coupled with the spatial complexity of traversing scene triangles reaching the $O(nm)$ level, and considering modern pursuit of resolution, its performance is unacceptable. However, traversing the triangles simultaneously yields specific information about them, allowing for further modification of geometric information. For modern pipelines, compute shader can be used to leverage the GPU's ray tracing cores for computation, but considering the resource consumption of modern applications, practical significance is limited.

On the other hand, the depth and normal-based approach, which introduces image space information and abandons the traversal of geometric information, offers the second-best performance. Individually, depth detection cannot detect regions with gradual depth changes, and normals cannot detect parallel edges, while the combined geometric result is correct. The naive method cannot obtain specific geometric information for further modification, but this problem is perfectly solved by modern pipeline. Furthermore, it has the highest compatibility with modern pipelines and is extremely compatible with shader writing logic.

The image analysis method, which only analyzes the image space, offers the best performance, but the problem is that the results are inaccurate. While it boasts $O(n)$ time and space complexity, it cannot avoid distortion caused by insufficient information. Modifying the filtering core places excessive demands on the designer. In terms of implementation approach, its adaptation to post-processing steps has some flexibility, but overall, it is the worst choice.

2.1.2 SHADING

The earliest shading methods can be traced back to Gooch's [13] parametric generative shading. It generates a 0-1 mapping table, detects lighting and the angle between the viewpoint and the object, and then performs shading to display geometric features. Sloan then realized that this mapping table could be pre-defined, manually creating mapped textures, allowing NPR to perform complex shading mappings. Benard then proposed a paper that adjusted offline shading weights based on object geometry [14], while maintaining the shading mapping range abruptly caused by high frame rate viewpoint movement. Later, *Guilty Gear*'s company [15] published NPR based on the PBR rendering pipeline, abstracting and subtracting normals, thresholds, and indirect lighting pollution. Finally, in recent years, implicit shading methods based on neural networks have also been proposed.

From the above, we can observe that there are four approaches to shading: parametric generation, Look Up Table (LUT) generation, geometry-based generation, PBR-based, and neural rendering. The implementation of neural rendering is similar to that of machine learning in the current industry, and will not be discussed here. Parametric generation undoubtedly offers the best space and time efficiency. Compared to LUTs and geometric methods, which rely on parameters to determine color without depending on textures or geometric calculations, the latter two depend on scene size for performance. In terms of result quality, LUTs offer higher detail than geometric methods, followed by parametric

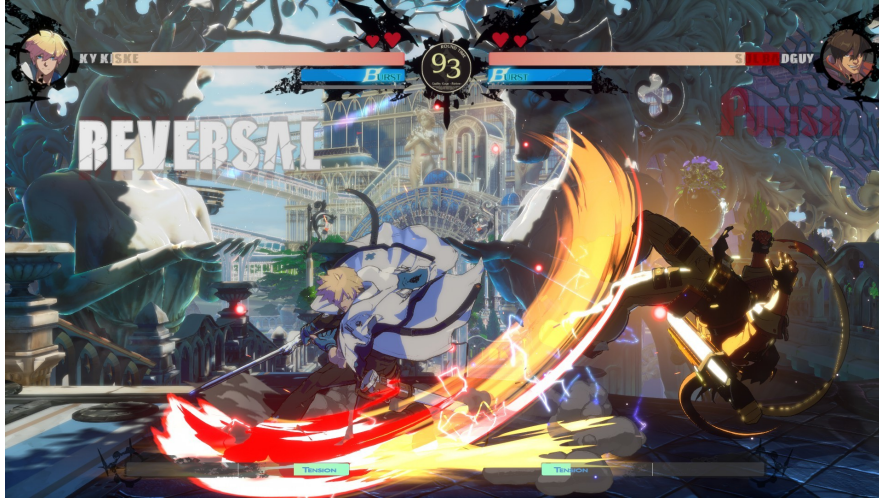


FIGURE 2.2: Guilty Gear

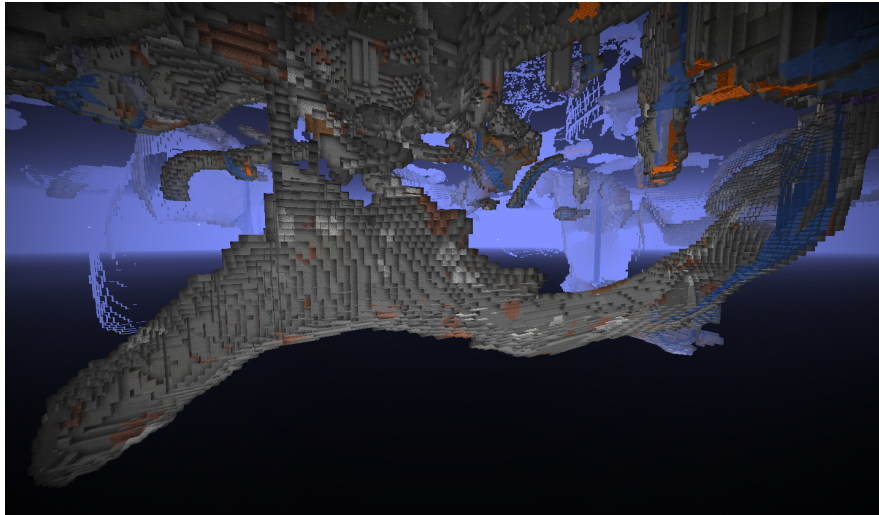


FIGURE 2.3: Minecraft's cave

methods. Regarding controllability, considering the difficulty and workload for designers, LUTs are the best, followed by parametric methods, with geometric methods being the worst.

PBR-based methods are relatively unique, offering the lowest performance due to their reliance on multiple textures such as normals and placements, requiring additional calculations for each render compared to the aforementioned methods. However, they introduce realistic textures, which are difficult to achieve with the other methods. In terms of controllability, it can compete with LUTs, but the disadvantage might be the significant additional workload required from designers. It is the best in terms of pipeline compatibility, as modern pipelines are themselves developed based on PBR principles.

2.2 PCG

2.2.1 PARAMETRIC METHODS

Lindenmayer, in his research [16] on biological cell growth models, proposed a mathematical model called L-System: a series of parallel initial states are set up, allowing them to grow sequentially according to simple rules, ultimately revealing a regular and valuable pattern—using symbolic growth to simulate the regular branching characteristics of reality, which can be extended from cells to branches and most plants. Perlin discovered the value of the continuous properties of a function in images [6]; the continuous transformation of random fields simulates the continuous features of reality and can be used for water surface and terrain simulation. Then, Stiny discovered patterns in 2D shapes [17], continuously selecting, expanding, and subdividing within a limited 2D range, with repetitive features in the overall and detailed shapes, similar to the way human architecture and city shapes are constructed. Barnsley then discovered the regular self-repeating features of geometry in reality [18]; snowflakes and petals grow according to this logic.

The precise mathematical definitions of noise (Perlin's) and fractal (Barnsley's) methods lead to accurate outputs, while L-systems and shape subdivisions require precisely defined rules to ensure error-free operation. Shape methods (Stiny's), due to their high dimensionality, are difficult to control. Controllability is the opposite, as the range of configurable rules actually increases. Finally, as the simulated real-world features cover a wider range, human architecture and cellular vegetation growth have broader applicability, while the rule-based simulations of the other two have fewer applicable scenarios. However, practical use still depends on the specific scenario.

2.2.2 RULE BASED

The rule-based approach aims to produce discrete results, rather than the output of continuous mathematical formulas generated by parameters.

Grammar-based methods can be seen as an extension of L-systems. Dormans gained inspirations from *The Legend of Zelda: The Twilight Princess* [19], outlining a series of switching rules from start to intermediate stage to finish, using random or weighted selections until a finish is reached, suitable for rogue-like levels. Johnson proposed a method in a 2D grid, progressively updating the cell by judging the state of its surrounding 8 cells according to rules until stability [20]. The results closely resemble the distribution of real-world caves, and *Minecraft*'s natural caves are based on this theory [2]. Londono formally uses graph algorithms, combined with extended rules, to fit non-linear, connected levels, such as *Super Mario Bros* [21]. Smith and Gillian et al. then uses a method with global constraints, dynamically and iteratively matching and adapting rules for generation [22], suitable for object visits and story sequences. Smith Adam et al. uses a series of verbs to define the rules to be completed, generating levels by obtaining solutions through various solvers [23], suitable for strategy and puzzle games like mazes.



FIGURE 2.4: The Legend of Zelda: The Twilight Princess



FIGURE 2.5: Super Mario Bros

In the five methods, the solver method yields the most accurate result, guaranteeing correct values. Next is the cellular automaton method, although its cave features may be disconnected. Graph methods and grammar methods are similar, but the former has unlimited length, while the latter has self-looping issues that require manual avoidance, and finally, the system method cannot guarantee complete content at the end of iteration, is ranked at the final. For designers, the system-bound method is the most acceptable one, followed by the grammar method and the solver method, then the graph method, and finally the cellular automata, ranked by their friendliness to designers. Ultimately, assuming correct results, the graph method, system method, and solver method support the highest level of descriptive detail, but also produce the most complex results. The grammar method is weak in spatial expressiveness, while the cellular method is limited to cave descriptions and cannot simulate other scenarios.

DISCUSSION

The review objectives are:

- Time and space complexity and effect quality (NPR) / Degree of accuracy of the result quality (PCG)
- Artists' controllability (Both).
- Pipeline compatibility (NPR) / result diversity and expressiveness (PCG)

3.1 GAP AND LIMITATION

NPR and PCG have different focuses, but also some similarities, such as the early Berlin noise technique and Gooch's parametric texture generation concept. NPR simplifies the logic of reality, while PCG deepens and simulates the logic of reality. Both belong to the art of approximation and pursue stylized creative generation.

While gap and limitation can be observed as well. As for NPR, in academic field, it shows that QI concepts' advantage are the ability of acquiring triangle's data, while in modern pipeline, this problem has been solved by shader. While, shading methods have all been utilized in real games, such as *Genshin Impact* [24] with LUT, *Arknights:Endfield* [25] with PBR, and *Hi-Fi Rush* [26] with geometric methods, choice is more depended on game design.

Regarding PCG, overall practical choice should rely more on use case than NPR's choice. Shape-based method is average best in parametric type, but shows only little advantage. In rule-based methods, solver method is the best, while it is worth noting that it is a offline generation method with highest entry barrier, thus readers need to make further judgments based on actual gameplay.

3.2 POTENTIAL DIRECTIONS

For NPR, modern pipelines offer controllable optimizations, and shader graph lower the barrier of the anticipation for designers. It is challenging though, future advancements in neural rendering technologies, such as DLSS 5.0[27], could potentially break through existing barriers if it enable designers to modify neural networks using abstract methods and achieve real-time inference within games.

**FIGURE 3.1:** Genshin Impact**FIGURE 3.2:** Arknights:Endfield

For PCG, techniques' choice are more dependent on the use case. Researchers still need to find methods to fit more potential real-world patterns and even complex ones. While machine learning and agent-based methods are not yet mainstream, they hold immense potential.

From an industrial practice perspective, both PCG and NPR involve significant designer involvement, such as style and rule definition. For NPR, many complex designs remain difficult to implement simply using existing processes. For PCG, the more complex the real-world model, the harder it is to involve designers. Developing related tools is a direction with significant practical potential.



FIGURE 3.3: HiFi Rush

4 CONCLUSION

Although NPR and PCG are two different domains, their origins reveal some overlap in research and shared creative concepts. However, they differ fundamentally in their development directions and priorities. This paper, based on academic exploration of the fundamental concepts of these two fields, proposes possible approaches and directions for industrial practice in both areas. It also provides a preliminary comparative analysis, highlighting comparative advantages and potential directions, offering a detailed reference for novice industrial developers to understand, trace, and explore future possibilities in these two fields.

BIBLIOGRAPHY

1. Santella, A. & DeCarlo, D. *Visual interest and NPR: an evaluation and manifesto* in *Proceedings of the 3rd international symposium on Non-photorealistic animation and rendering* (2004), 71–150.
2. Shaker, N., Togelius, J. & Nelson, M. J. Procedural content generation in games (2016).
3. Appel, A. *The notion of quantitative invisibility and the machine rendering of solids* in *Proceedings of the 1967 22nd national conference* (1967), 387–393.
4. Strassmann, S. Hairy brushes. *ACM Siggraph Computer Graphics* **20**, 225–232 (1986).
5. Mitsa, T. & Parker, K. J. Digital halftoning technique using a blue-noise mask. *Journal of the Optical Society of America A* **9**, 1920–1929 (1992).
6. Perlin, K. An image synthesizer. *ACM Siggraph Computer Graphics* **19**, 287–296 (1985).
7. Togelius, J., Yannakakis, G. N., Stanley, K. O. & Browne, C. *Search-based procedural content generation* in *European Conference on the Applications of Evolutionary Computation* (2010), 141–150.
8. Doran, J. & Parberry, I. Controlled procedural terrain generation using software agents. *IEEE Transactions on Computational Intelligence and AI in Games* **2**, 111–119 (2010).
9. Summerville, A. *et al.* Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games* **10**, 257–270 (2018).
10. McReynolds, T., Blythe, D., Grantham, B. & Nelson, S. Advanced graphics programming techniques using OpenGL. *Computer Graphics*, 95–145 (1998).
11. Zhang, H. & Hoff III, K. E. *Fast backface culling using normal masks* in *Proceedings of the 1997 symposium on Interactive 3D graphics* (1997), 103–ff.
12. DeCarlo, D. & Santella, A. Stylization and abstraction of photographs. *ACM transactions on graphics (TOG)* **21**, 769–776 (2002).
13. Gooch, B. & Gooch, A. A. 10 Interactive Non-Photorealistic Rendering.
14. Bénard, P. *et al.* Stylizing animation by example. *ACM Trans. Graph.* **32**, 119–1 (2013).

15. Motomura, J. C. *GuiltyGearXrd's Art Style : The X Factor Between 2D and 3D* in *Game Developers Conference* <https://gdcvault.com/play/1022031/GuiltyGearXrd-s-Art-Style-The> (San Francisco, CA, USA, 2015).
16. Lindenmayer, A. Mathematical models for cellular interactions in development I. Filaments with one-sided inputs. *Journal of theoretical biology* **18**, 280–299 (1968).
17. Stiny, G. & Gips, J. *Shape grammars and the generative specification of painting and sculpture*. in *IFIP congress (2)* **2** (1971), 125–135.
18. Barnsley, M. F. *et al.* *The science of fractal images* (Springer, 1988).
19. Dormans, J. *Adventures in level design: generating missions and spaces for action adventure games* in *Proceedings of the 2010 workshop on procedural content generation in games* (2010), 1–8.
20. Johnson, L., Yannakakis, G. N. & Togelius, J. *Cellular automata for real-time generation of infinite cave levels* in *Proceedings of the 2010 Workshop on Procedural Content Generation in Games* (2010), 1–4.
21. Londoño, S. & Missura, O. *Graph Grammars for Super Mario Bros Levels*. in *FDG* (2015).
22. Smith, G., Whitehead, J. & Mateas, M. *Tanagra: A mixed-initiative level design tool* in *Proceedings of the Fifth International Conference on the Foundations of Digital Games* (2010), 209–216.
23. Smith, A. M. & Mateas, M. Answer set programming for procedural content generation: A design space approach. *IEEE Transactions on Computational Intelligence and AI in Games* **3**, 187–200 (2011).
24. Cai, H. *'Genshin Impact': Crafting an Anime Style Open World* in *Game Developers Conference* <https://gdcvault.com/play/1027539/-Genshin-Impact-Crafting-an> (San Francisco, CA, USA, 2021).
25. Wikipedia contributors. *Arknights: Endfield* — *Wikipedia, The Free Encyclopedia* [Online; accessed 9-April-2026]. 2026. https://en.wikipedia.org/w/index.php?title=Arknights:_Endfield&oldid=1343970046.
26. Kosuke Tanaka, T. K. *3D Toon Rendering in 'Hi-Fi RUSH'* in *Game Developers Conference* <https://gdcvault.com/play/1034251/3D-Toon-Rendering-in-Hi> (San Francisco, CA, USA, 2024).
27. Lin, H. *NVIDIA DLSS 5 Delivers AI-Powered Breakthrough In Visual Fidelity For Games* <https://www.nvidia.com/en-gb/geforce/news/dlss5-breakthrough-in-visual-fidelity-for-games/>. Online; accessed 3-April-2026. 2026.